

Speaker Notes, Slide by Slide · MHE-NMPC Payload-Adaptive Control for a Drone

Assumed length: 15 minutes (16 slides, about 55 seconds each). To cut it to 10 minutes, drop slides 5, 9, 13 and 14, and fold the real-time numbers from slide 6 into slide 5.

General rules:

- One idea per slide. Once that idea is across, move on; do not read the slide aloud.
- Whenever something was **not** done or **not** shown, say so yourself. What reviewers care about most is whether you know where your own limits are.
- Say numbers as integers or with one decimal ("thirty-two out of thirty-six", not "eighty-eight point nine percent").

01 · Title — *Eventless Payload-Adaptive NMPC* (about 30 s)

Good afternoon. My talk is about payload-adaptive control for a drone. The scenario is a delivery drone that picks up and drops a parcel in flight.

The figure on the right is Fig. 3 from the paper, so let's look at it first: the blue line is the estimated payload "first mass moment", the green dashed line is the ground truth. It rises on its own at pickup, holds through the figure-8, and returns to zero after the drop.

The key point: at no time does any "gripper closed" or "parcel released" signal reach the estimator. The ground truth is only used for offline comparison; the drone itself never sees it. That is the problem this work solves.

Don't: open with textbook definitions of MHE and NMPC. First let them see that it really estimates the payload.

02 · Problem — *Grasping a parcel breaks the model the controller is using (about 55 s)*

Why this is hard. Picking up a parcel changes three things at once.

First, total mass changes, so the thrust-to-acceleration map changes. The hover throttle is wrong, and the drone sinks or overshoots. That is an outer-loop problem.

Second, the center of mass shifts. Thrust no longer passes through the composite CoM, which creates a **constant trim torque**. Note the scale: thrust is roughly mg , which is large, so **even a small offset produces a significant torque**. That is an inner-loop problem.

Third, inertia increases, so the same torque turns the drone more slowly, which effectively lowers the inner-loop bandwidth. Also an inner-loop problem.

But what really motivated this paper is a fourth point, and it is the first sentence of the abstract: **a release command does not guarantee that the parcel has actually left**. A command is only an intention. The gripper may not open, the object may fall late, or the command may never arrive. A controller that clears its payload model on the command will fly a drone that is still carrying cargo as if it were empty.

This slide anchors the whole talk. If time is short, cut later slides rather than rush the fourth point.

03 · Architecture — *The pipeline consumes only motor speeds and odometry (about 55 s)*

This is Fig. 1 of the paper, the information flow of the whole system.

The green line at the bottom is the key: **the entire chain consumes only two things — motor speeds and odometry**. No force sensor, no external motion capture, no "payload attached" signal.

In the top left there is an "attach / drop notification" with a no-entry sign, labelled NOT CONSUMED. That is not a drawing shortcut. **It is the core constraint of this work:** we deliberately keep that signal out of every path.

The ATOMIC FRAME in the middle is the only interface between the MHE and the NMPC. One frame carries mass, first moment, inertia increment, a health bit and the solution age together.

On the right, the NMPC uses it to update its model and to gain-schedule PX4's rate loop. If the evidence is insufficient, it takes the UNRESOLVED path at the bottom and brakes smoothly to a hover.

If asked "why must it be one frame": see the card on this slide. Split across several topics, the NMPC could receive combinations that do not exist physically, such as a new mass with an old inertia.

04 · Contribution 1 — *Estimate the first mass moment, not the CoM offset (about 70 s, worth the extra time)*

This slide is the most important modelling choice in the paper, so I'll spend time on it.

Intuitively, if the CoM shifts, we should estimate the CoM offset r_{xy} . But that breaks at the moment of release. Physically, the first moment s equals payload mass times offset, so it naturally goes to zero when the parcel is gone. **The offset r_{xy} itself, however, becomes 0/0 as the payload mass goes to zero — it is simply undefined.**

This is not a theoretical worry. In an early mass-only version, the only way the optimizer could suppress a non-existent offset was to push the total mass down, below the empty drone. After release, 14 of 55 samples were pinned at the lower bound.

Estimating the first moment s fixed it. The first moment is the standard linear parameterization of rigid-body inertial parameters, a classic tool in manipulator payload identification. And with s and m , the CoM offset follows by division — **without knowing anything about the grasp geometry.**

In the paper we describe it as the **minimal structure** we found that expresses three things at once: whether the payload is present, how large the trim torque is, and the roll/pitch inertia. The inertia increment ΔJ is not estimated independently; it is generated algebraically from the mass estimate and a weak prior on the vertical lever arm — a lever arm that is structurally unobservable from the thrust-torque channel.

This slide is the most likely to draw questions; see Q4 and Q5 in the anticipated questions.

05 · NMPC — *Re-solving one second of future every 50 milliseconds (about 45 s)*

The controller. The essential difference between NMPC and PID is that NMPC does not react to the current error. It uses a nonlinear model to predict one second ahead, solves for an optimal input sequence within the actuator envelope, applies only the first step, and re-solves next cycle.

20 Hz, one second of look-ahead. Median solve time is 1.3 ms, 99th percentile 2.2 ms — 4.4% of the 50 ms control period.

One thing I want to stress on the right-hand card: **the hover reference is live.** The mass in the input reference is the online mass estimate, and it **includes the trim torque produced by the offset**; it is not a hard-coded empty-drone weight. Otherwise, once the payload changes, the cost function keeps pulling thrust back to the old hover value.

Payload information enters the model as **runtime parameters**, so there is no code change and no solver regeneration.

06 · MHE — *Payload parameters become extra states (about 55 s, includes a weakness you must raise yourself)*

The estimator. MHE looks at the last two seconds and finds the states and parameters that make the model's prediction best match the measured trajectory. Because it uses more than the current frame, it can use information across time to suppress noise.

The state is 16-dimensional: the 13-D vehicle state, plus mass, plus the two components of the first moment.

The weights are deliberate: **process-noise weights are much tighter than measurement-noise weights**. That says "apart from the unknown payload parameters, the model structure is right". So the optimizer can only explain the observed acceleration by adjusting mass and first moment; it cannot vaguely push the error onto process noise.

Conversely, the arrival cost is deliberately **almost uninformative** — the standard deviation on mass is 3.2 kg. We don't use a strong prior, because with a strong prior the estimate is just an echo of the prior.

There is one issue I have to raise myself: **the MHE's timing tail is narrow**. The 9.2 ms median is fine, but the 99th percentile is 59 ms and the peak is 72.8 ms, against a 100 ms period. It meets its deadline in simulation, **but the paper explicitly makes no claim about onboard compute margin** — that has to be measured on the target computer.

This part earns credit. Reporting the timing tail yourself is far better than having it pulled out of you.

07 · Contribution 2 — *Never feed the controller's own thrust command back in (about 60 s)*

The second key decision is about where the MHE's input forces come from.

The most convenient source is the thrust command the NMPC itself sends. **But that is a trap.**

The NMPC's cost anchors thrust near "estimated mass times g ". As soon as the mass estimate is off, the commanded thrust no longer equals the force the drone actually feels. Worse, the inversion error between command and rotor, saturation, and motor lag are **all invisible to the estimator**. So a mass error can persist, self-consistently, without ever showing up as a residual — the estimator ends up confirming itself.

Our approach is to take the measurement **downstream** of all of that: we reconstruct physical thrust and physical torque directly from motor speeds. That signal has passed through the real nonlinear maps of PX4 and the motors, and **is not computed from the mass estimate**.

One implementation detail is easy to get wrong: motor messages arrive much faster than the MHE period, and what must be averaged over the window is **each motor's thrust**, not the speed before squaring. Squaring is nonlinear; doing it in the wrong order introduces a systematic bias.

The paper also states what this depends on: a calibrated thrust coefficient k . Slide 15 gives the quantitative cost of getting it wrong.

08 · Contribution 3 — *Issuing the release command is not evidence (about 55 s)*

The third point is where the word "eventless" in the title lands: on what basis does the controller decide that the parcel is gone?

There is only one path: **confirmation**. On healthy, fresh estimate frames, the empty-payload confidence must exceed 0.90 and stay there for 1 second.

That confidence is the product of three scores: mass, first moment, and inertia.

The product means any one of them can veto on its own — even if the mass estimate has hit its lower bound, a non-zero first moment can still block the false call.

What if the evidence is insufficient? We don't guess. We enter UNRESOLVED: a 12-second timeout, then a 3-second smooth brake to a hover, and we collect evidence again at low dynamics. **A timeout is always treated as "unknown", never as proof that the drone is empty.**

The card on the right is a fast path we removed. It used "residual plus mass-domain empty", with no first-moment evidence. In the 4 m/s figure-8, 3 of 9 random flights released while the parcel was still attached. The reason is that under aggressive maneuvering, the mass estimate sits on its lower bound for whole stretches, so the mass domain reads "empty" for a long time.

The bottom line is the most convincing part: **one recorded figure-8 transient produced a residual of -1.69 N , larger than the -1.47 N of a real release of the smallest payload.** So residual magnitude alone cannot tell "maneuvering" from "actually dropped".

09 · Rigour — *Where every threshold came from (about 45 s)*

This slide is not about the thresholds themselves, but about where they **came from**.

The paper devotes a full table to the origin of every threshold and states that all of them were fixed in code before the final batch started — **none was re-tuned on that batch**.

Only two values were chosen against data: the first-moment collapse ratio and the residual floor. Both used **early flights disjoint from the final batch**. The collapse ratio of 0.30 came from leave-one-out cross-validation on 59 early flights, and every fold picked 0.30.

The rest are engineering choices, and the paper says plainly that "we did not sweep their closed-loop sensitivity".

The point I want to make: **where a threshold comes from is itself something that should be reviewed**.

If time is short, fold this slide into one sentence on slide 11. But if the examiners care about experimental methodology, this is the slide that earns the most credit.

10 · Experiment design — *One mission, four ways for the payload to be gone (about 55 s)*

Only checking that a normal drop is recognized doesn't prove much. The real test is when the command and the physics **disagree** — and that is the only place where "not relying on event signals" can actually be tested. So we designed four ways for the parcel to be gone.

A is the normal drop, the baseline mission.

B injects a 4-second delayed detach: the command is sent, but the physical connection holds. Any scheme that clears the model on the command must call it too early.

C is loss without a command: the parcel falls, **but no command ever reaches the controller**. A command-armed confirmer cannot detect this by construction — this is the one battleground where only we can win.

D is partial loss: three of four boxes fall one after another. It tests tracking of multi-level steps, not a binary decision.

Two points on method: the paired design was fixed **before** the comparison, not by picking an arm afterward; and the safety metric uses a separate large sample — 209 loaded flights, 3.7 hours, with **zero** false releases before the command.

11 · Main result — 32 of 36 — and what that does not establish (about 70 s, key slide)

The main result. Every flight where the payload physically attached and the drone survived to the release command is counted — 36 in total, and 32 were confirmed.

All four failures **were at 4 m/s, and all trace back to estimator failure**: three never recognized the payload at all, and one diverged after release. I'll come back to this in the limitations.

Confirmation latency is **bimodal**, and the paper refuses to average the two groups into one number. 17 flights confirmed during the figure-8, median 3.5 s; the other 15 could not get evidence during the maneuver, ran the full 12-second timeout and braked, median 16.6 s. These are two different control paths and cannot be merged.

The table on the right is the one I want to stress. **It is a self-limitation, not a score**: these 32 confirmations took **three different paths** — 19 moment-gated, 4 mass-domain, 9 on confidence alone. So the paper says that 32 of 36 is a conditional closed-loop result for **the whole interface, not a validation of any single release detector**.

How you say this slide matters: report the result first, then narrow its scope yourself. That is better than waiting for an examiner to narrow it for you.

12 · Baseline — *Head-to-head with clearing on the command (about 55 s)*

A direct comparison with the conventional approach. Arm A clears the payload model immediately on the command, which is today's usual practice; arm B is our interface.

Look at the first row: **under nominal conditions the two arms tie, 19 to 19, with no post-release instability in either.** So we **do not sell the nominal case**; that row is honest reporting, not a result.

The difference shows up in the second row. With a 4-second delayed detach injected, arm A cleared early in all 5 runs, a median of 4 seconds too early; arm B **never cleared early** in its 6 valid runs, and in 5 of them it went through UNRESOLVED before confirming.

But what really decides it is the card on the right. When no command ever reaches the controller, our interface detected 10 of 18, while the command-armed comparison arm **detected 0** and kept a phantom first moment the whole time.

I admit 10 of 18 is not pretty. But the difference here is **can versus cannot**, not good versus better.

Seven of the eight misses share one cause: when the loss happened, the loaded reference window had not formed yet, or the payload had never been recognized at all.

13 · Single flight — *What one complete flight looks like* (about 40 s)

What a complete flight looks like: 0.15 kg, 2 m/s, confirmed during the maneuver. On the left is the measured horizontal path: blue while carrying, orange after release, grey dashed is the reference.

On the right are three traces: altitude, **the mass estimate the controller actually consumes**, and the position-error norm. The mass rises on its own after pickup, holds through the figure-8, and drops back after release — with no external signal anywhere in between.

The 0.6 m error peak during the lift comes from the lift itself, not from payload identification. Tracking RMS is 0.053 m while loaded and 0.060 m after release, which is comparable.

This flight confirmed 4.46 seconds after the command. **That delay is a price we choose to pay**, in exchange for semantics that never run ahead of the physical evidence.

14 · vs. L1 — *Against the adaptive baseline: claims and non-claims (about 50 s)*

The comparison with the standard adaptive-control baseline, L1-adaptive NMPC. Three methods times four conditions, 5 runs per cell: all 60 runs were stable, zero divergence.

First, **what we do not claim**: transient peaks are indistinguishable across the three, within 0.05 m per cell. In two cells L1 is higher, $p = 0.008$ and 0.012 , **but those don't survive correction for eight comparisons, so we claim no transient advantage.**

What we do claim is recovery speed: L1 is 8% to 32% slower than ours, median 16%, significant in three of four cells. The mechanism is clear — L1's compensation low-pass cut-off is capped by closed-loop stability. On this platform 0.5 is stable and 1.0 is not; it is a bandwidth wall, and explicit estimation does not face that wall.

Two more things must be said together with this. First, **this table uses the early 14-state, mass-only MHE, not the final first-moment interface**, so it only supports a bounded baseline comparison. Second, L1 provides no interpretable physical parameters, and its gain scheduling still needs the same kind of prior.

15 · Limitations — *What does not hold yet (about 60 s, take this seriously)*

Every item on this slide is written in the paper itself.

First, all evidence is from simulation. Moving to hardware means replacing simulated motor speeds with ESC telemetry and re-identifying the thrust and torque coefficients, noise floor, timing and gripper compliance.

Second, and the hardest one: **the thrust coefficient must be calibrated to within 5%.** Scaling only the estimator's thrust map by $\pm 5\%$ made the interface fail in both directions — at 0.95 the payload was never recognized in 4 of 4 runs; at 1.05 a 0.09 kg phantom payload remained after release.

Third, the first moment is systematically underestimated: a median of 10.8% during carrying, same sign in all 31 runs. **We have not found the cause.** Fortunately the detector uses a ratio, so it is insensitive to this kind of common scale bias.

Fourth, in 13 of the 32 completed flights a qualifying reference window never formed, so the moment-ratio test never even ran.

Fifth, release is not gated on estimator health. In one flight the estimate had been unhealthy for 48 seconds before release, but the phase-timed release command went out anyway; after release the model was mismatched and the flight diverged.

Sixth, 0.3 kg at 4 m/s is a stress test, not an operating point. Hover uses only 74% of thrust, **but torque hits its limit first** — in the figure-8, pitch torque is saturated in 98% of the diagnostic windows.

Slow down on this slide. Explaining your own failures clearly builds more trust than reporting one more success number.

16 · Summary — *What stands, and what I did myself (about 50 s)*

Three sentences to summarize.

First, minimal sensing: the whole chain consumes only motor speeds and odometry; attach/drop notifications enter no path.

Second, the first-moment interface: payload geometry stays well-defined at the moment the parcel disappears.

Third, release semantics: the model is cleared only on sustained evidence; if evidence is missing, we enter UNRESOLVED and brake.

Next steps are the move to hardware and bench calibration of the thrust coefficient, gating the release command on estimator health, and adding the missing baselines.

Finally, one thing I want to state up front. **This project used AI assistance for code generation, refactoring, adding tests and organizing documentation.** I am responsible for the experimental design, the choice of criteria and the conclusions. My current focus is on systematically understanding and verifying the generated results. In this talk I distinguish three kinds of content: what has been confirmed by code and tests, what holds only in simulation, and what still needs checking.

One example: the Chinese README still says "the MHE is diagnostic only and does not feed back into the controller", while in the code the estimates have long been in the NMPC loop. **I found that inconsistency while checking** — and it also shows that reading the documentation is no substitute for reading the code.

Don't read this part; say it naturally. Explaining the division of work proactively, concretely and with evidence earns more trust than glossing over it.